

eGalvanic SLD v3 — Critical Bug Findings

Release-gate sign-off · deep interactive bug hunt

Date	2026-06-10
App under test	https://acme.qa.egalvanic.ai/slds (v1.21, GoJS diagram engine)
Tester	Claude Code (Opus), driven by abhiyant.singh@egalvanic.com
Method	Live Playwright (non-headless) + GoJS-model introspection + parallel unauthenticated API sweep (8 agents) + direct curl auth probes
Scope	Create / move / delete / edit / export of SLD diagrams; three-layer integrity (UI / browser-local / backend); security (XSS, auth/BOLA)

Verdict: SLD v3 is **not release-ready** — a non-functional Export, a create flow that can only produce orphan (unconnected) nodes, the S1/S2/S6 data-integrity cluster (87 of 107 SLDs), and a 2× render cost are all open. Security & stored-XSS, however, tested **clean**.

1 Critical / HIGH scoreboard

ID	Severity	Status	Summary
CRIT-1	HIGH	NEW	Web "+ Asset" creates orphan nodes; no way to draw edges (live S1 cause)
SLD-BUG-15	HIGH	confirmed	Export button does nothing (release blocker)
SLD-BUG-14	HIGH	confirmed	Diagram double-mounted → 2× render + duplicate fetches
BUG-G S1/S2/S4/S6	HIGH	confirmed	Orphans / default-coord pile-up / unclassified edges / soft-delete leak (87/107 SLDs)
SLD-BUG-09	HIGH	confirmed	Wild Goose: 490 nodes / 0 rendered edges
SLD-BUG-17..20	MED	confirmed	Delete "cannot be undone" vs Undo; dead Delete-key; drag moves neighbor; a11y focus-trap
Stored <script> exec	SAFE	tested	Does NOT execute (React-escaped)
Unauth API / BOLA	SAFE	tested	All SLD read+write endpoints 401 on both hosts

2 CRIT-1 HIGH NEW — Web "+ Asset" creates structurally-orphan nodes

The web SLD editor can **add** and **move** nodes, but **cannot draw edges** (`diagram.allowLink === false`, and there is no connect / link tool anywhere in the UI). So **every node created in the web is born with 0 connections, and there is no in-web way to ever connect it**. This is the live, current-code root cause of systemic bug **S1** (orphan nodes, 82 of 107 SLDs) — not merely a legacy migration artifact.

Steps to reproduce

1. Open `https://acme.qa.egalvanic.ai/slds` and pick site **gyu** in the *Site* selector.
2. Click the red **Lock Graph** FAB (bottom-right) once to **unlock** editing.
3. In the top toolbar click **+ Asset** → the **Select Node Type** palette opens (Fig. 2.1).
4. Click any type, e.g. **Fuse** → a banner shows **"Click to place asset: Fuse"** (Fig. 2.2).
5. Click an empty area of the canvas → a node **"Fuse-3"** is created and persisted (`POST https://eg-pz.qa.egalvanic.ai/api/node/create` → **201**).
6. Inspect the new node's connections — it has **0 links** (Fig. 2.3), and there is **no tool to connect it** to anything.

Expected: a newly created device can be wired into the single-line diagram (the whole point of an SLD). **Actual:** the node is permanently isolated; the web offers no edge-drawing affordance, and the on-canvas **"No issues"** badge does not flag the disconnected node.

Impact (HIGH): every web-created node becomes an orphan → directly produces the S1 connectivity-loss cluster. A user "building" an SLD in the web ends up with a pile of unconnected symbols. *(The node does land at the clicked coordinate, not (100,100) — so this is NOT the cause of S2's pile-up, which comes from mobile/import. The create also logs an `iOS bridge not available for handler: graphUpdate` dead-call.)*

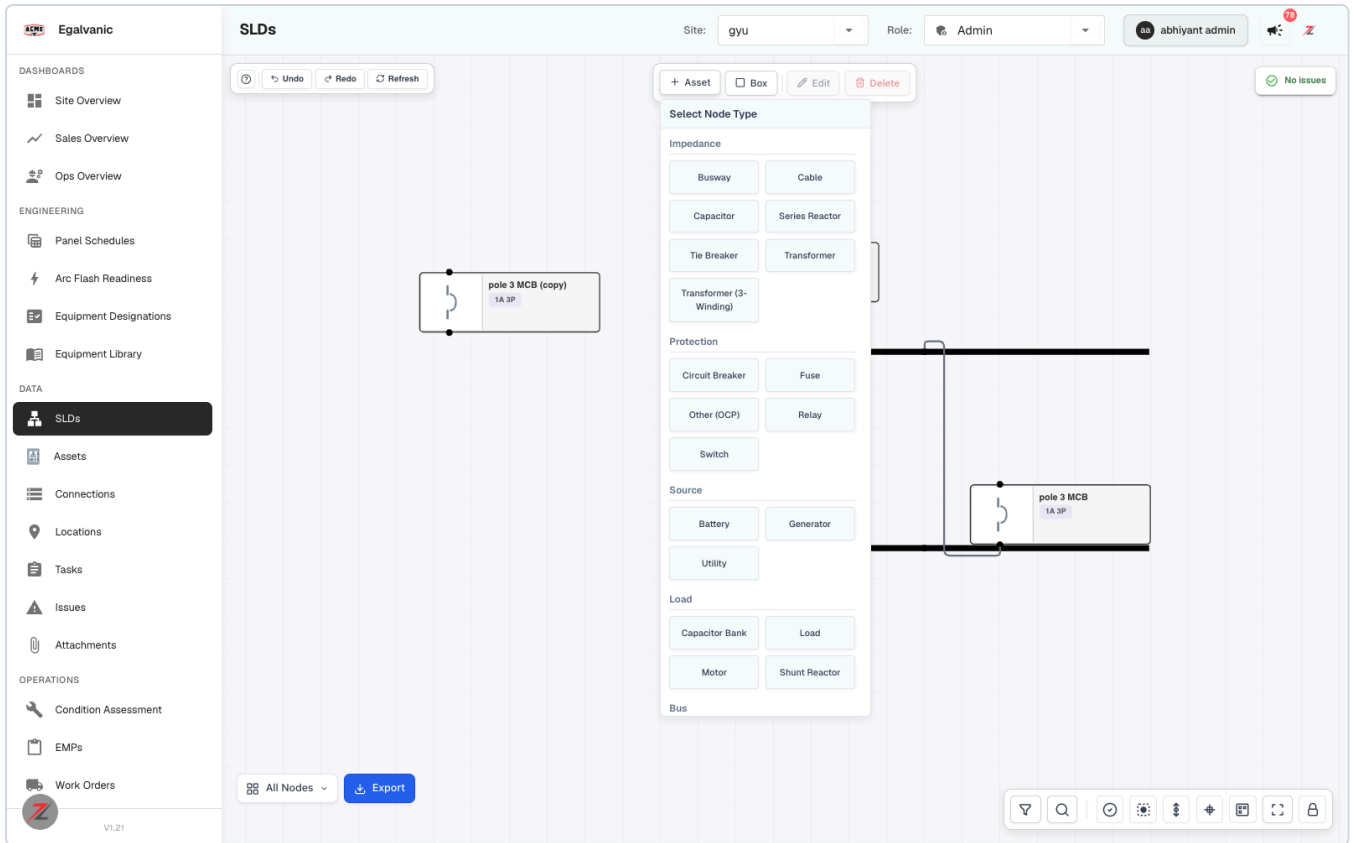


Fig. 2.1 — +Asset opens the “Select Node Type” palette (Impedance / Protection / Source / Load / Bus).

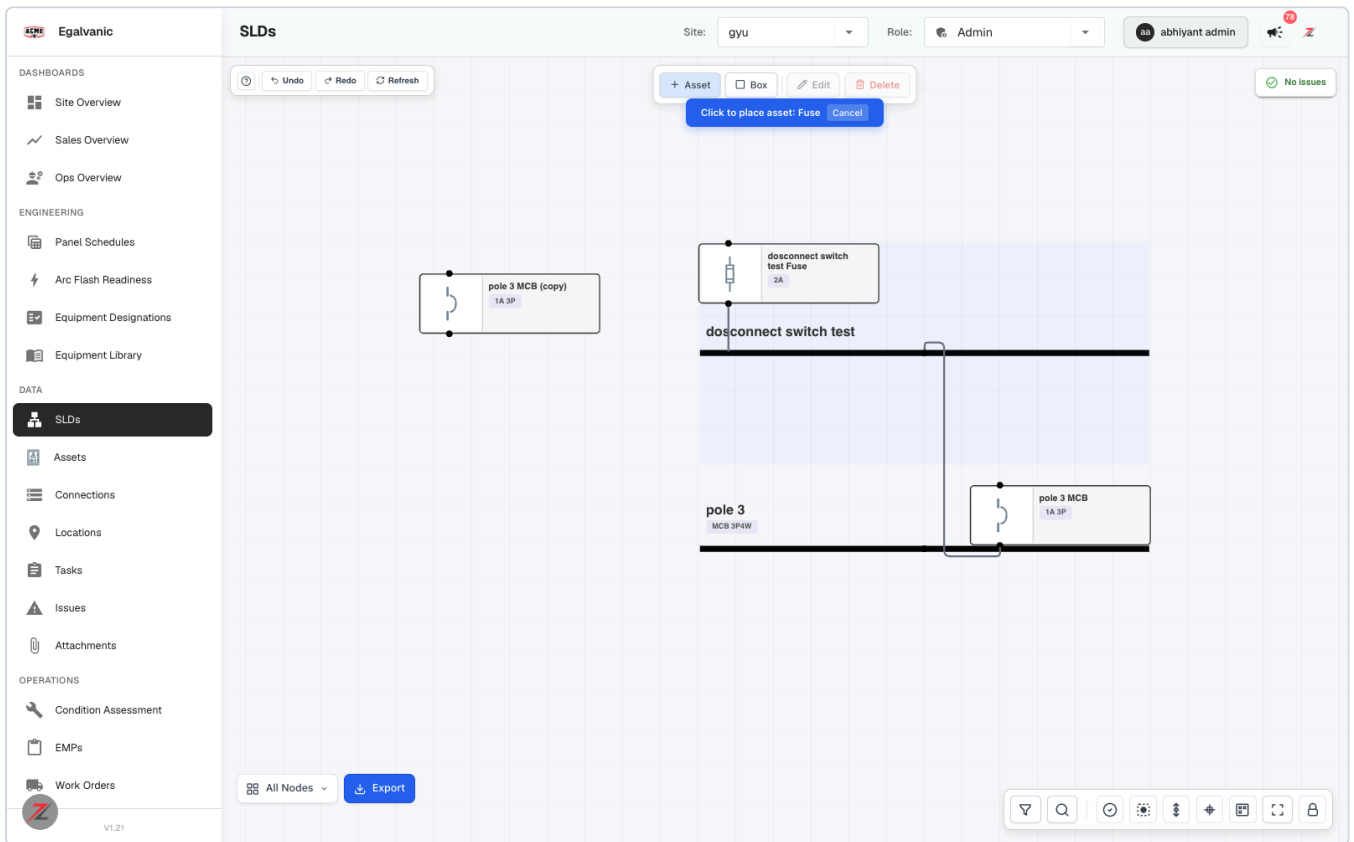


Fig. 2.2 — “Click to place asset: Fuse” placement mode.

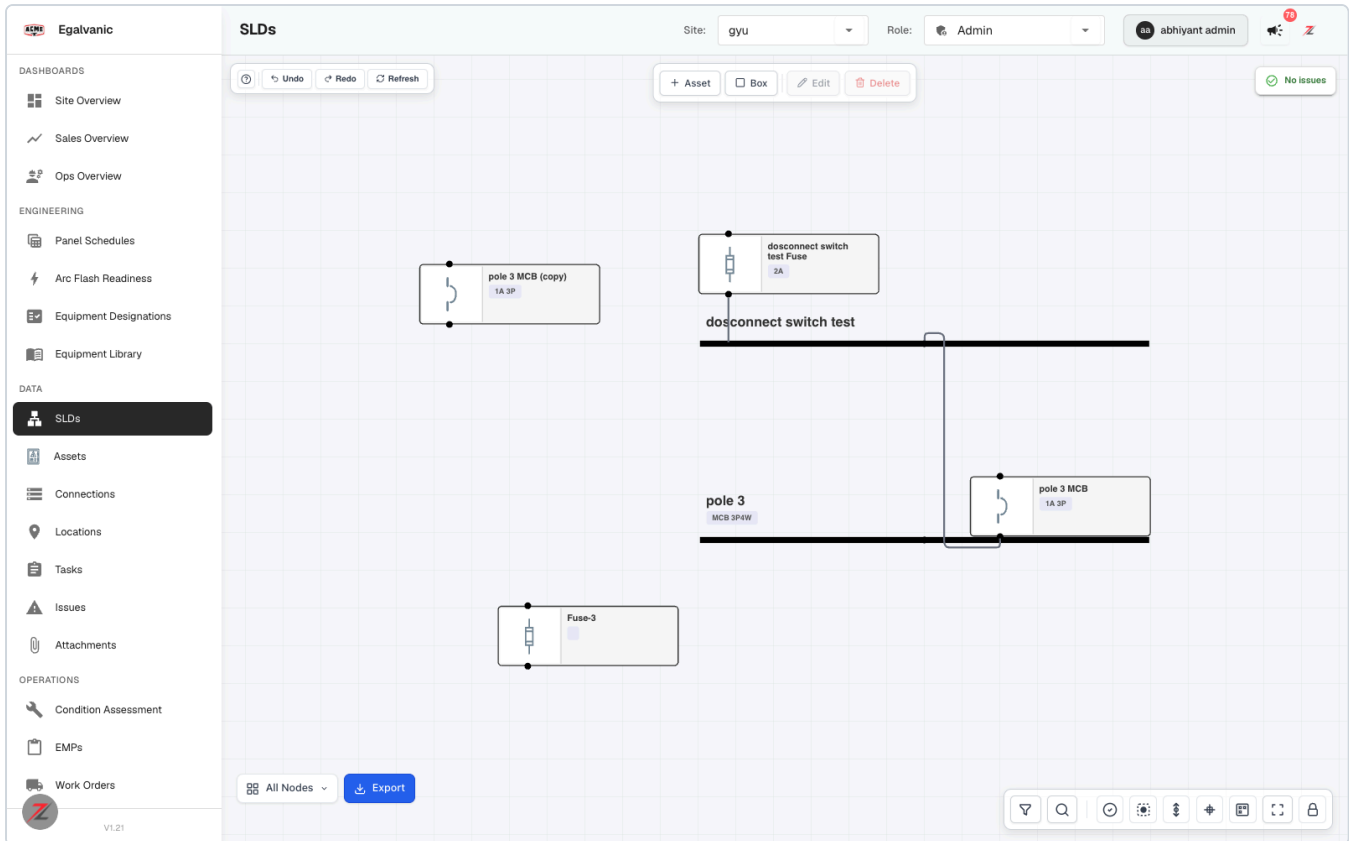


Fig. 2.3 — the created node “Fuse-3” (bottom-left, selected) is fully disconnected: 0 edges, and no way to connect it.

3 CRIT-2 / SLD-BUG-15 HIGH — Export is a silent no-op (release-gate blocker)

Steps to reproduce

1. Open any SLD (e.g. gyu).
2. Click the blue **Export** button (bottom-left of the canvas).
3. Observe the result.

Expected: an exported PDF / PNG / JSON of the diagram (explicit v3 release-gate capability). **Actual: nothing happens** — no file download, no format menu, no dialog, no network request, and no console log or error. Reproduced across 3 attempts.

Impact: the “export diagrams” feature is effectively non-functional and gives the user zero feedback.

4 SLD-BUG-09 / S1 HIGH — 490 nodes render with 0 edges (connectivity lost)

Switching to (s) **Wild Goose Brewery** renders **490 nodes and 0 edges** — the single-line diagram shows no connections at all. The backend keeps only 5 “active” edges and all 5 are orphans (their endpoints reference deleted nodes), so GoJS draws none. This also exposes **SLD-BUG-14**: the page mounts **two** GoJS diagrams (one in a `display:none 0x0` container) and **both** fully load all 490 nodes → 2× render cost + the duplicate-fetch storm.

Steps to reproduce

1. Open `/slds`; in the *Site* selector choose (s) **Wild Goose Brewery**.
2. Wait ~8s for the diagram to load.
3. Observe: hundreds of device boxes, **no connecting lines** between them (Fig. 4.1).
4. Introspect the GoJS model: `diagram.nodes.count = 490`, `diagram.links.count = 0`; and `G.Diagram.fromDiv` returns **two** diagrams both with 490 nodes.

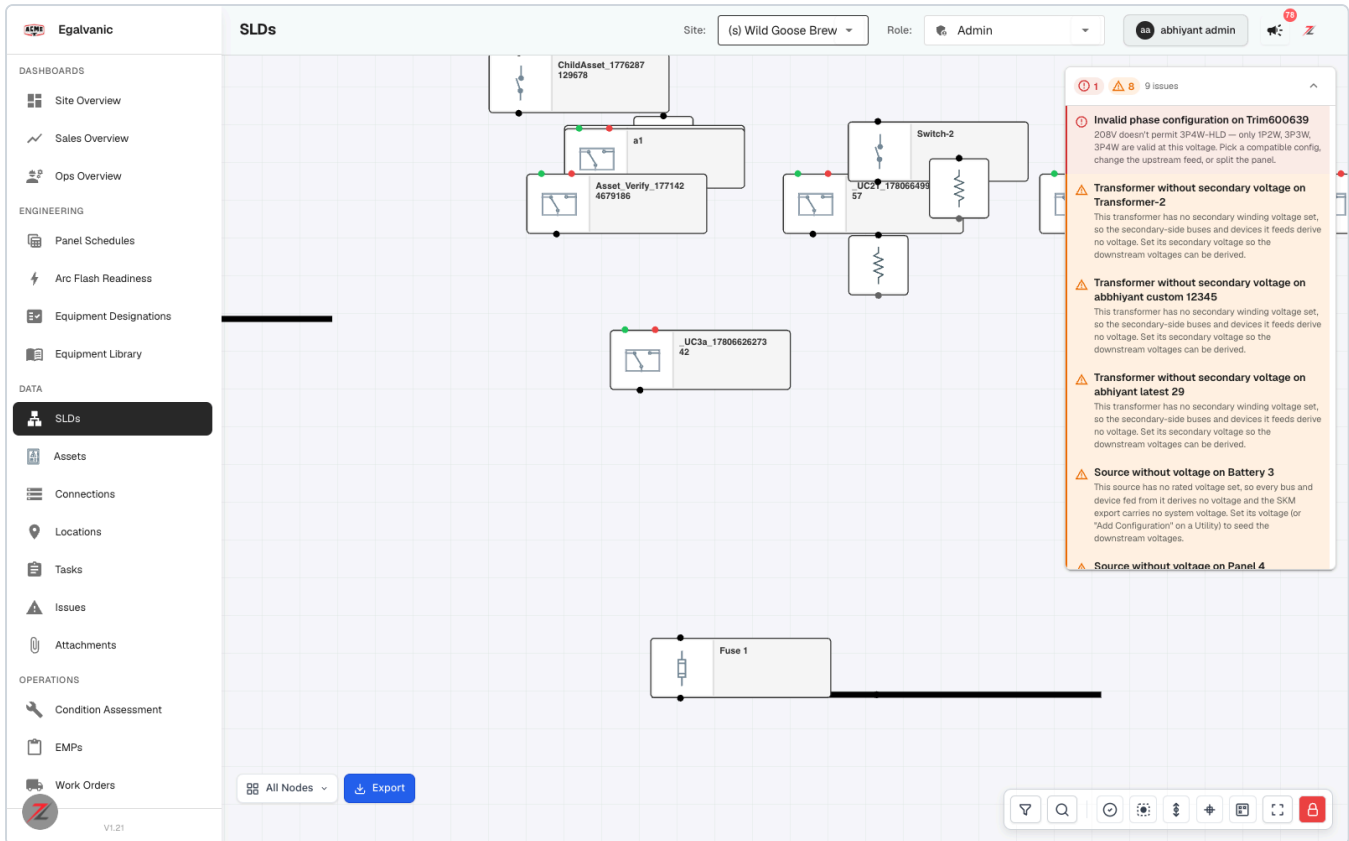


Fig. 4.1 — Wild Goose: 490 nodes, 0 edges. The right-hand panel lists real electrical issues, but the topology is gone.

5 SLD-BUG-01/06 / S2+S7 MED — Default-coordinate pile-up & label overlap

On the small **gyu** SLD, 4 of 6 nodes sit at exactly $(100, 100)$ and render stacked on top of each other; bus-group labels overlap and become unreadable, yet the badge still says “**No issues**” (validation checks electrical data, not layout). With the v3 view toggles on (MiniMap, Trace Lineage, Edge Labels, Status Badges) the overlap is vivid (Fig. 5.1).

Steps to reproduce

1. Open `/slds` → site **gyu**.
2. Enable the toolbar toggles: **Show MiniMap**, **Trace Lineage**, **Show Edge Labels**, **Show Status Badges**.
3. Observe the overlapping bus group “dosconnect switch test” colliding with “pole 3” / the fuse, while the badge reads “No issues”.

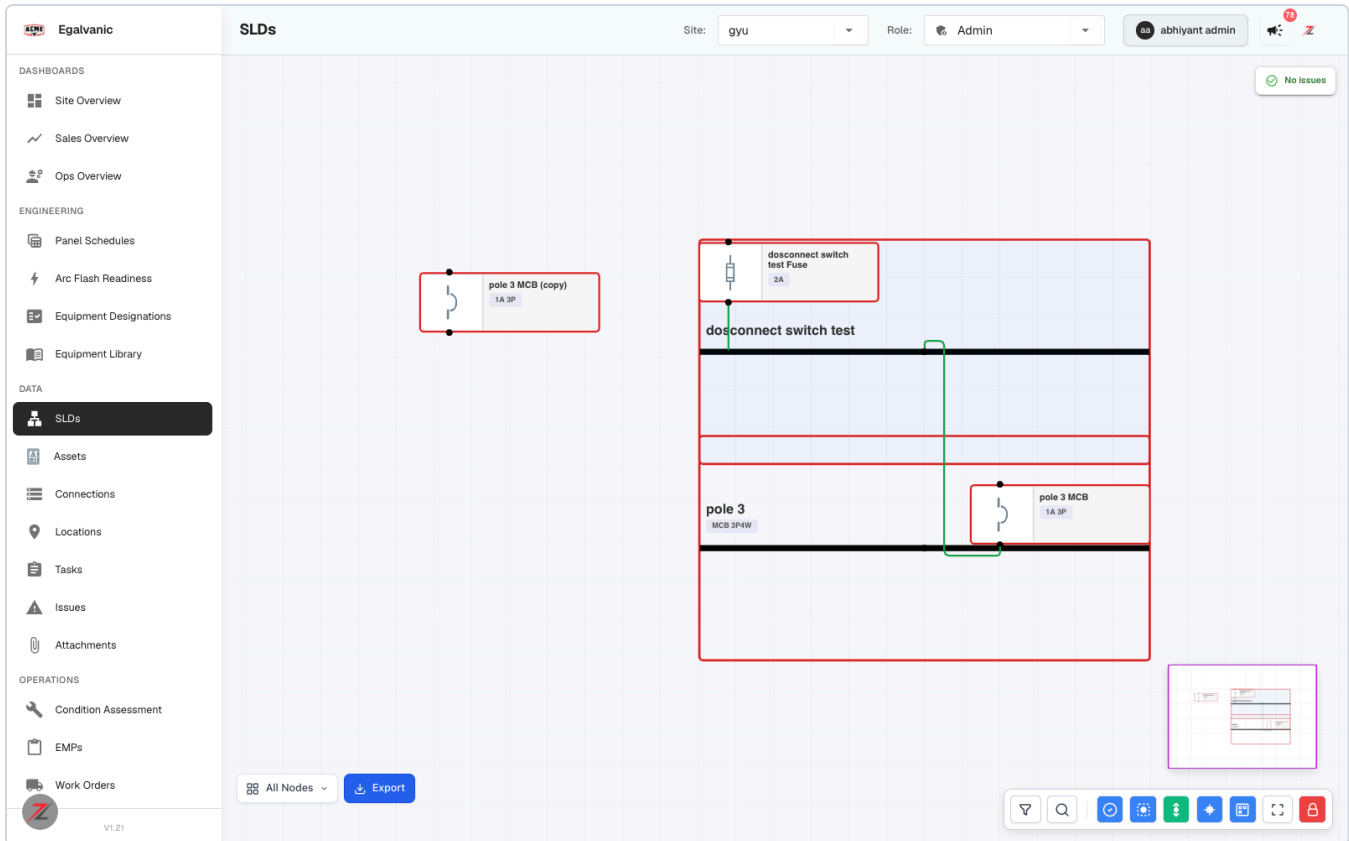


Fig. 5.1 — v3 view features active; overlapping/garbled bus labels while the badge claims “No issues”.

6 Edit affordances, delete semantics, and the stored-XSS check

Unlocking + selecting a node reveals an editing toolbar (Undo / Redo / Refresh / Copy / Add to... / + Asset / Box / Edit / Delete).

Findings:

- **SLD-BUG-17** MED — the Delete confirmation says “**This action cannot be undone**” while the toolbar exposes an **Undo** button. Delete is a soft-delete (`POST /api/node/bulk-delete` → 200) whose records keep leaking back through `/api/sld/{id}` (= S6) — contradictory and a data-loss-confusion risk.
- **SLD-BUG-18** MED — the **Delete** key and **right-click** do nothing; deletion is only possible via the toolbar button.

Stored-XSS test — SAFE (does NOT execute)

Wild Goose contains a node literally named `<script>alert('XSS')</script>_1780930307763`. Tested for execution:

1. Site selector → **(s) Wild Goose Brewery**; wait for load.
2. Override `window.alert` and scan the DOM for any live `<script>` / `<img onerror>` carrying the payload.
3. Select the payload node and open **Edit** → the **Edit Asset** dialog renders the name in an `<input>` (Fig. 6.1).
4. Result: `alert` never fires; 0 injected scripts; canvas is plain GoJS text.

Verdict: not exploitable as stored XSS (React default escaping + input-value rendering). Residual is data-hygiene only: the field stores raw `<script>` (no write-time sanitization). **Do not file as XSS.**

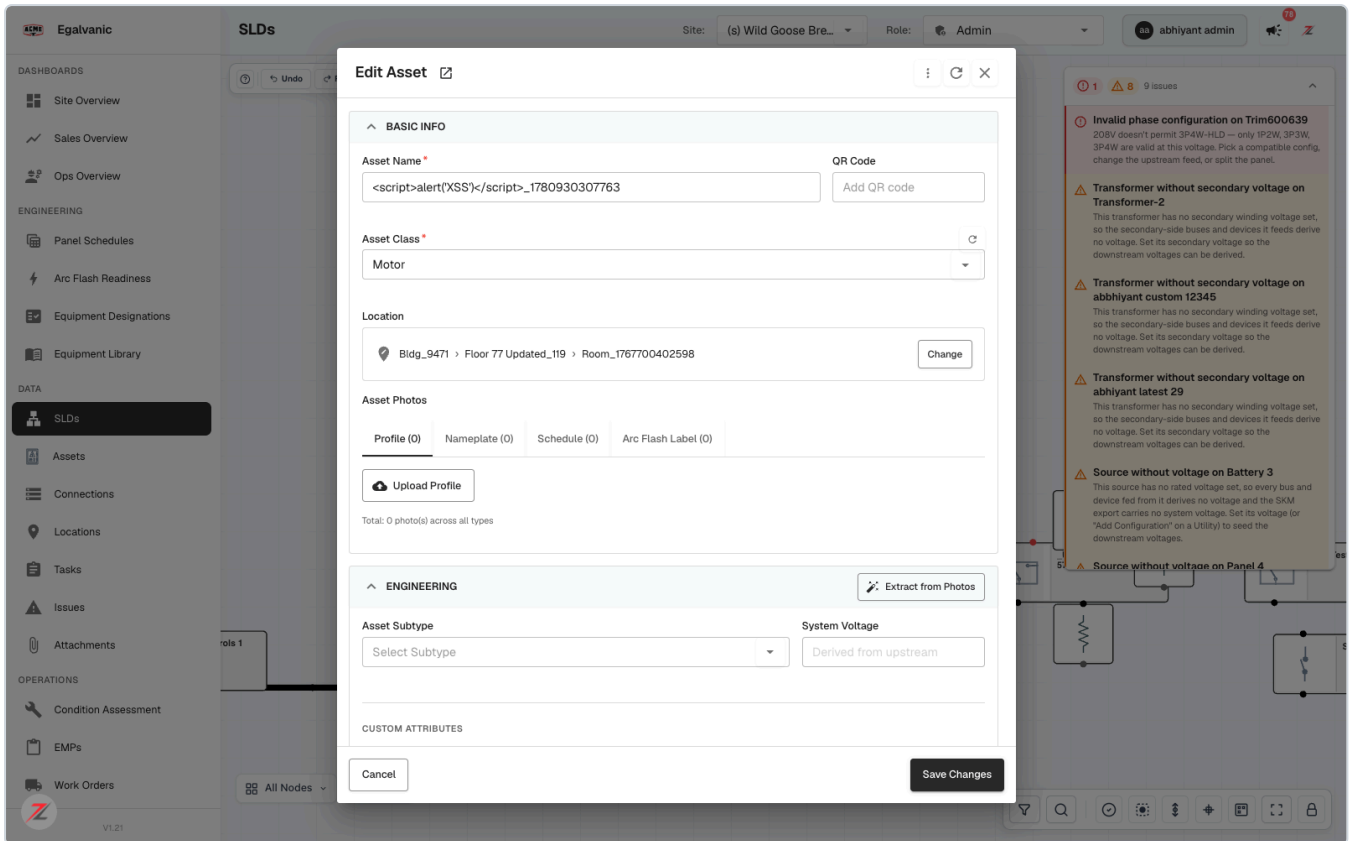


Fig. 6.1 — the `<script>` payload sits inertly in the Edit Asset **Name input field** (escaped, not executed).

7 Security — VERIFIED SAFE (no unauthenticated data exposure / BOLA)

A parallel unauthenticated sweep (8-agent workflow) plus direct `curl` with **no token/cookie** against the real SLD endpoints on **both** API hosts:

Endpoint	Method	Unauthenticated result
acme.../api/sld/{realId}	GET	401 "No authorization provided"
acme.../api/lookup/nodes/{realId}	GET	401
acme.../api/users/{realId}/slds	GET	401
acme.../api/node/update/*, edge/update/*	PUT	401
eg-pz.../api/node/create, /node/update/*, /node/bulk-delete	POST/PUT	401 "Missing or invalid authorization header"

Verdict: SLD read **and** write endpoints correctly reject unauthenticated requests on both hosts — **no BOLA / data-exposure**. The only quirk: unmatched `/api/*` paths return **200 with the SPA index.html** instead of a 404 JSON (the same SPA-fallback routing behind BUG-F's `JSON.parse` error storms).